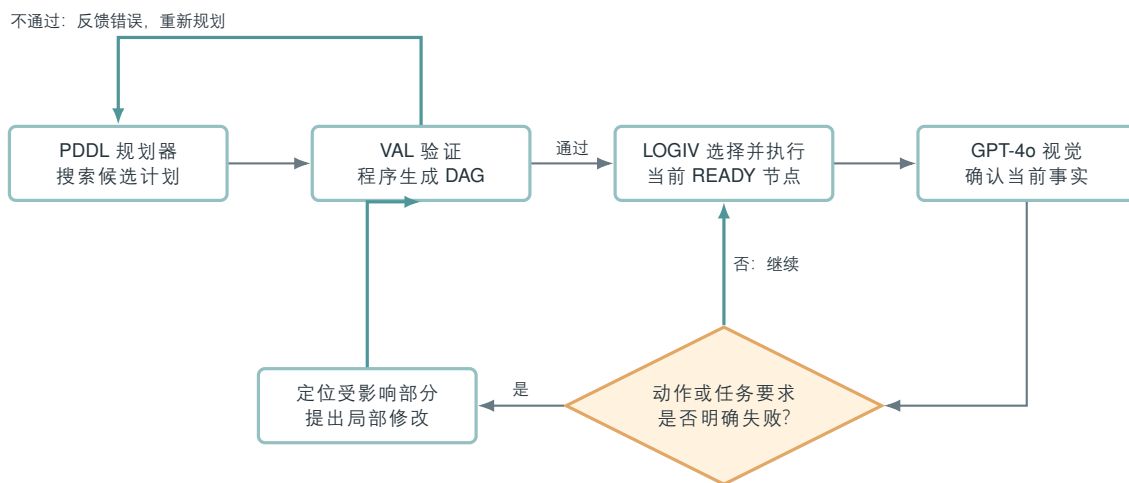


LOGIV：全程认证的任务图执行与局部修复

从初始 DAG 到节点级执行、在线检查与局部修复

动机与方法概览 顺序计划通常只规定下一步做什么，却难以同时表达并行分支、节点依赖和已完成结果；执行中一旦状态变化，整段重规划也容易丢失仍然有效的工作。LOGIV 用 DAG 保存这些关系，并从任务开始实时调度：PDDL 符号规划器搜索初始计划，VAL 验证后由程序建图；节点开始前和结束后调用视觉模型确认事实。明确失败出现后，符号规划器只重规划受影响的节点及其下游依赖，验证通过后从当前状态继续执行。



四个 Gate 如何放入主线 State Gate 负责确认视觉事实；Node Gate 检查单个动作；Graph Gate 检查整张任务图；Repair Gate 在明确失败后组织局部修复。它们是一个闭环中的四项职责，不是四套彼此独立的算法。

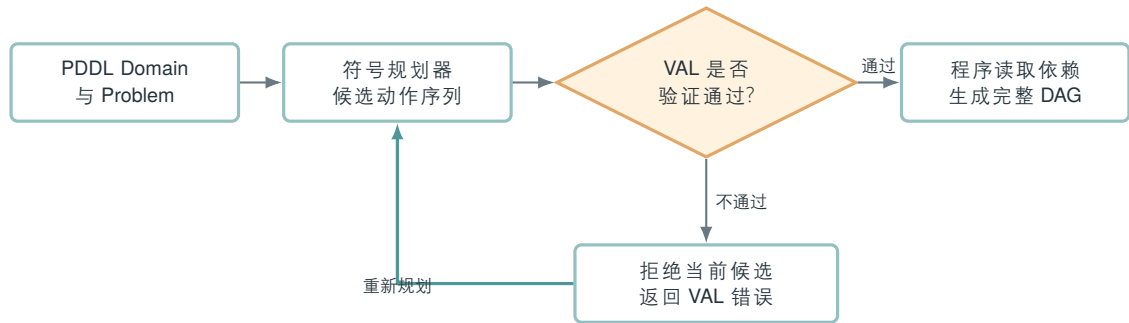
两条基本规则 VLM 在首个节点派发前、每个节点开始前和结束后调用；仅在 UNKNOWN 时补充新帧或新视角，不在每个低层 step 重复运行。符号规划器只输出 PDDL 动作序列，DAG 的节点与依赖边仍由确定性程序生成。

阶段一

符号规划器生成计划，VAL 验证计划能否执行

PDDL 定义搜索空间；规划器搜索动作；VAL 独立验证。

PDDL Domain 在运行前定义合法的高层动作、参数类型、前置条件和效果；Problem 给出注册对象、当前事实和最终目标。符号规划器在这个有限搜索空间中寻找从初始状态到最终目标的动作序列。候选只能使用已定义动作和已注册对象，不包含机器人连续控制量。



符号规划器负责什么

根据当前 PDDL 状态、动作前置条件和目标搜索动作序列；修复时只对允许修改的范围重新搜索。

符号规划器不负责什么

不增加 PDDL 动作或对象，不修改最终目标，不直接画 DAG 的边，也不输出低层控制量。

为什么还需要 VAL

规划器输出仍需独立检查，因为模型转换、Problem 构造或规划结果可能存在参数、前置条件和目标可达性问题。VAL 对完整计划进行验证；一次不通过只拒绝当前候选，系统修正当前 Problem 或搜索约束，并在有限次数内重新规划。

DAG 不是语言模型生成的 通过 VAL 的具体动作构成 DAG 节点。若动作 A 的效果是动作 B 的前置条件，程序加入从 A 到 B 的依赖边；若两个动作存在资源或事实冲突，程序加入必要顺序。因此，相同的合法计划会得到相同的任务图。

阶段二

State Gate：把图像转换成可使用的当前事实

视觉模型读取事实，State Gate 判断证据是否足够可靠。

系统使用独立的 GPT-4o 视觉调用回答封闭问题，例如“moka pot 是否位于炉区”。模型不判断系统是否失败，也不提出修复；它只针对系统指定的事实返回三值结果。



State Gate 输出	含义	后续处理
PASS	观察明确、没有过期，并与当前对象和任务图一致	将事实交给 Node Gate 或 Graph Gate
UNRESOLVED	图像不足、结果冲突，或对象与版本无法确认	不修复；获取新图像并重新观察
FAIL	可靠观察与当前任务记录明确冲突，且不属于动作或任务图级失败	记录状态偏离，暂停当前节点并进入 Repair Gate

为什么需要 UNKNOWN

UNKNOWN 表示“不知道”，不是“事实为假”。如果图像被遮挡或模型无法区分对象，LOGIV 暂停节点派发，并依次获取新的时间帧、其他相机或视角，以及预先注册的安全观察动作。每次重试必须增加新证据；有限尝试后仍然无法判断时，系统安全停止，而不是猜测状态或进入 Repair Gate。

State Gate 的边界 它只回答“这条观察能不能使用”。前置条件不满足、动作结果没有出现以及最终目标不成立，分别由后续的动作级或任务级检查负责，避免同一个现象被重复分类。

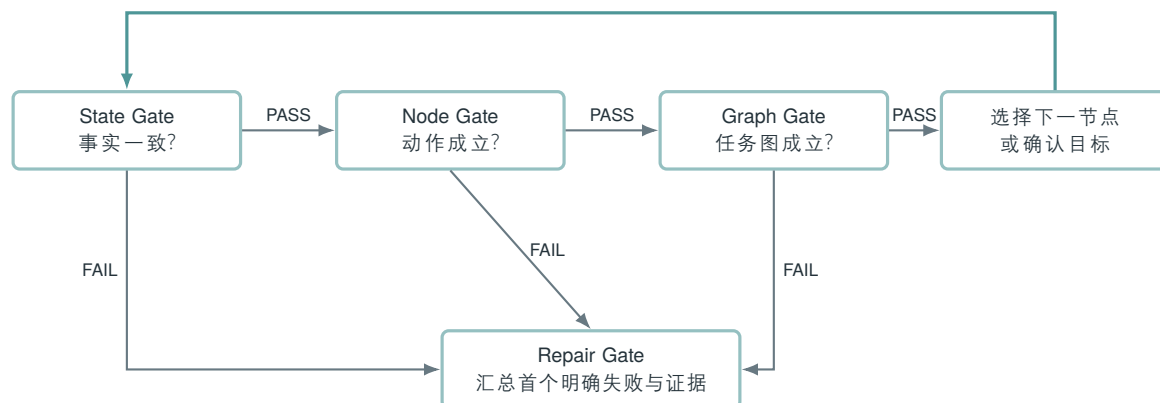
关键区分 UNRESOLVED $\neq$ FAIL；只有明确失败才进入 Repair Gate

阶段三

三个 Gate：任一明确失败都进入 Repair

依次检查当前事实、当前动作和整张任务图。

三个 Gate 都能独立发现明确失败：State Gate 检查当前事实能否与任务记录一致；Node Gate 检查当前动作能否开始、正常执行并产生预期结果；Graph Gate 检查已完成结果、整体进度和最终目标。任意一个 Gate 返回 FAIL，都暂停当前节点并进入同一个 Repair Gate；只有 PASS 才继续下一层检查。



检查层级	检查内容	明确失败时记录什么
NODE GATE	动作前置条件、执行过程、动作效果	前置条件失败、执行失败或效果失败
Graph GATE	已完成结果、整体进度、最终目标	已完成结果被破坏、任务停滞或目标失败

失败只记录一次 对同一次观察或动作，系统只记录最先被明确违反的要求。随后出现的相关现象作为补充证据，不再生成多个并列的主要失败。这项修复要求决定 Repair Gate 可以修改哪些节点。

证据不足时不执行 无论在动作前还是动作后，只要必要事实仍是 UNKNOWN，系统就不执行新动作，也不把节点标记为完成。证据不足不会被包装成失败。

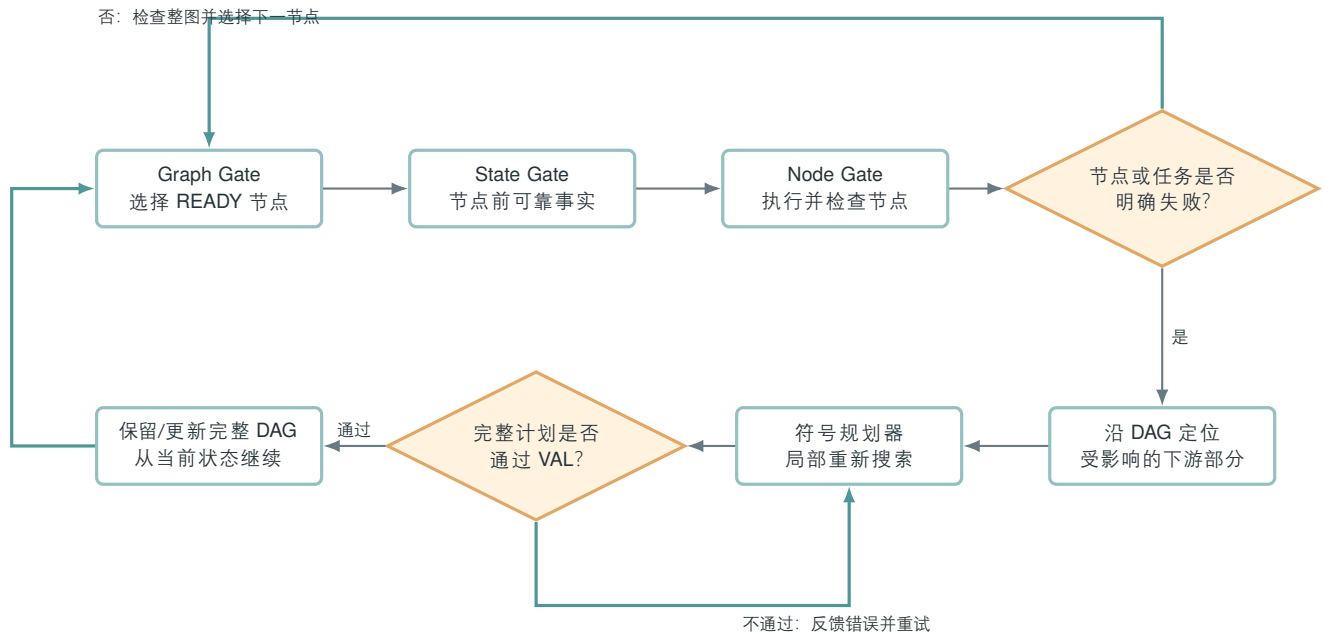
检查顺序 可靠状态 → 当前动作 → 整体任务

全程调度与修复

LOGIV 从第一个节点开始调度，失败后只修改受影响部分

DAG 决定执行顺序；原冻结机器人策略只负责节点内部的连续控制。

初始计划通过 VAL 并生成 DAG 后，LOGIV 选择第一个 READY 节点。VLM 在三类时刻观察：首次派发前建立初始事实；每个节点开始前确认前置条件；节点结束后确认动作效果并更新任务图。节点内部的低层 step 只检查原生成功、预算、执行器异常和紧急停止；出现 UNKNOWN 时才额外获取新帧或新视角。



局部修改与完整新图并不矛盾

Repair Gate 先用失败证据定位一个“失效边界”：失败节点、依赖其效果的下游节点，以及因资源冲突而必须重排的节点可以修改；已完成且仍成立的节点、与失败无依赖关系的并行分支和最终目标保持不变。符号规划器以当前可靠状态为新起点，在这个范围内重新搜索满足剩余目标的 PDDL 动作序列。程序把结果合并回原计划，VAL 检查完整计划；通过后再按 PDDL 的前置条件和效果重建依赖边，确认新图仍无环，才一次性安装新版本 DAG。进入 Repair Gate 后，一定会重新评估当前 DAG 是否仍然有效；只有旧 DAG 不能从当前状态继续完成任务时，才生成并安装新的 DAG 版本。

Repair Gate 的闭环 记录失败与当前事实 → 沿依赖边确定修改范围 → 符号规划器局部重规划 → VAL 验证完整计划 → 程序重建并检查 DAG → 从当前状态重新选择 READY 节点。验证不通过则调整 Problem 或搜索约束后重试；证据持续未知、规划无解或预算耗尽则安全终止。

贯穿示例

moka pot：从全程节点调度到局部图修复

同一个例子说明初始 DAG、四个 Gate、验证和继续执行。

初始计划要求把两个 moka pot 放到炉区。pot_1 已经完成；pot_2 原本应从左侧初始位置开始，但执行过程中落到了登记过的桌面恢复区。



修复前后

	修复前	修复后
内容		
当前事实	pot_2 被计划记录在左侧初始位置	可靠视觉确认 pot_2 位于恢复区
受影响动作	place-on(pot_2, left, stove)	place-on(pot_2, recovery, stove)
保持不变	pot_1 的已完成结果与最终目标	相同内容继续受到保护
图的产生	初始计划通过 VAL 后由程序生成	局部修改合并并通过 VAL 后由程序更新
执行方式	原机器人策略执行旧图节点	同一策略从当前状态执行新图节点

读者应当记住的核心 LOGIV 从初始 DAG 的第一个 READY 节点开始全程调度。初始计划与局部修复都由 PDDL 符号规划器完成；GPT-4o 只作为视觉模型读取指定事实。State、Node 和 Graph Gate 提供明确的修复依据，VAL 和确定性建图程序阻止未经验证的修改进入执行。

本文暂时隐去的实现细节 具体错误码、版本字段、重试次数、预算数值、迟到消息处理和完整终止状态表属于实现细节，不影响上述方法主线，可在后续论文版本中放入附录。